

CSC209 Assignment 3 Project Report

(AI Usage: The original draft was written by me and the ideas are all my own. ChatGPT 5.4 was used to improve clarity and flow of the writing.)

5.1 Project Overview

This project is a Category 1 multi-process application that uses pipes to distribute text-file analysis across a fixed pool of worker processes. The executable, 'analyzer', accepts one or more file names on the command line and starts a controller process that forks three workers at startup, matching the default configuration defined in protocol.h at line 6. Program execution begins in main() at main.c:13, which calls run_controller() at controller.c:481. The controller is responsible for assigning files to idle workers, receiving completed results, and producing a final summary for the user through functions such as dispatch_pending_tasks() at controller.c:326, collect_ready_results() at controller.c:399, and print_summary() at controller.c:105.

The purpose of the application is to analyze multiple input files concurrently and report the number of lines, words, and characters in each file. Each worker repeatedly receives a task message containing a filename, opens and analyzes that file, and sends a structured result message back to the parent through a dedicated pipe. This worker-side loop is implemented in worker_main_loop() at worker.c:45, while the per-file processing step that builds each reply is handled by process_task_message() at worker.c:14. The core file-counting logic itself is implemented in analyze_file() at analysis.c:22. This design allows several files to be processed at the same time instead of analyzing them sequentially in a single process.

From the user's perspective, the program is a command-line tool rather than an interactive application. The user runs 'analyzer' with a list of input text files, and the program prints status messages as tasks are assigned and completed. Task assignment messages are printed from dispatch_task_to_worker() at controller.c:292, and completed results are incorporated in mark_task_completed() at controller.c:350. For each successfully processed file, the controller reports the counts returned by the worker; if a file cannot be opened or read, the controller reports an error for that specific file. After all tasks are complete, the controller prints a final summary showing how many files succeeded, how many failed, and the total number of lines, words, and characters across all successful analyses in print_summary() at controller.c:105. This makes the tool useful both as a demonstration of concurrent systems programming with pipes and as a simple parallel text-analysis utility.

5.2 Build Instructions

The project is built using the provided Makefile in the a3 directory. Running make with no arguments compiles the full program and produces the executable analyzer. The required build command is:

```
make
```

If the executable or object files from an earlier build should be removed first, the repository also provides the command:

make clean

After cleaning, the project can be rebuilt with:

make

To run the program, execute analyzer followed by one or more input text files. The general command format is:

```
./analyzer file1.txt file2.txt file3.txt
```

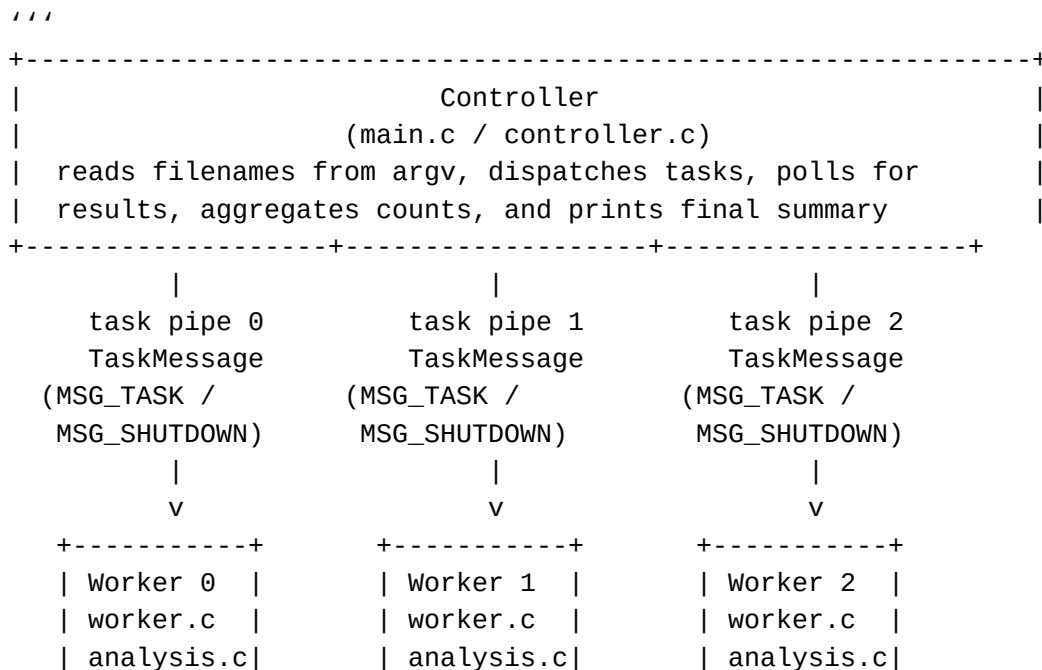
For example, using the sample files included with the project:

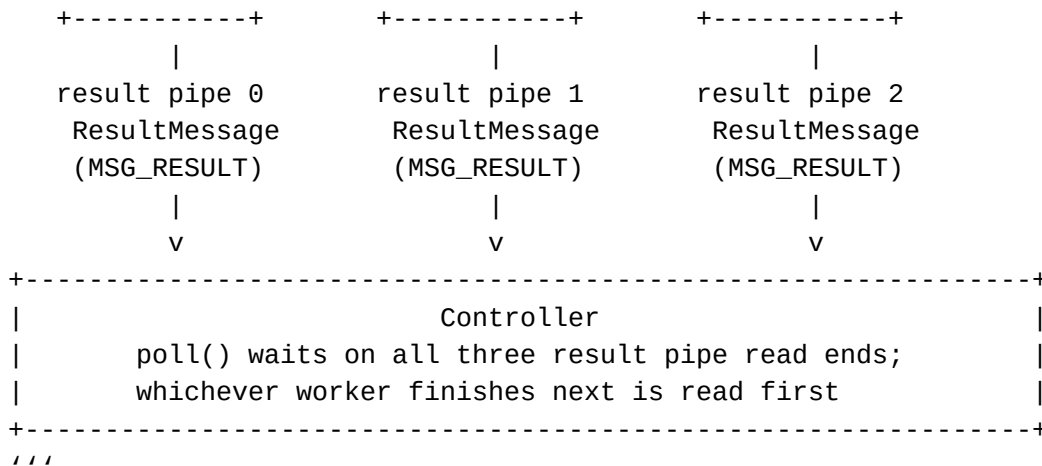
```
./analyzer demo1.txt demo2.txt demo3.txt
```

The project requires input files because each worker is assigned filenames to analyze. The repository includes demo1.txt, demo2.txt, and demo3.txt as sample inputs for testing and demonstration. No external libraries, configuration files, or additional command-line options are required. The project is written in C and builds with the standard compiler settings defined in the Makefile.

5.3 Architecture Diagram

The diagram below shows the three concurrently running worker processes and the two dedicated pipes that connect each worker to the controller. The controller creates and connects these pipes in `spawn_worker()` at `controller.c:182`. The controller holds the write end of each task pipe and the read end of each result pipe; each worker holds the complementary ends. All other pipe ends are closed immediately after forking inside `spawn_worker()` at `controller.c:182`.





5.4 Communication Protocol

The message types are identified by the type field in each struct. The valid values are MSG_TASK, MSG_SHUTDOWN, and MSG_RESULT, defined in the MessageType enum at protocol.h:10-13.

Message Type: MSG_TASK

Encoding: A fixed-size TaskMessage struct written in binary form in a single write_full call. The struct contains an integer type field, an integer task_id field, and a fixed-size filename array.

```
typedef struct {
    int32_t type;
    int32_t task_id;
    char filename[512];
} TaskMessage;
```

Semantics and invariants: This message assigns one file-analysis task to a worker. The controller constructs and sends it in `dispatch_task_to_worker()` at `controller.c:292` by setting `type` to `MSG_TASK`, setting `task_id` to identify the logical task being tracked by the controller, and copying a null-terminated filename into the `filename` field. The worker reads exactly one `TaskMessage` from its task pipe in `worker_main_loop()` at `worker.c:45`, checks that `type` is `MSG_TASK`, and then analyzes the named file. The filename must fit within the fixed-size filename buffer; if it does not, the controller marks the task as failed locally instead of sending a malformed message. After a valid `MSG_TASK` is received, the worker must produce exactly one corresponding `MSG_RESULT` message through `process_task_message()` at `worker.c:14`.

Error handling: If `write_full()` fails while the controller is sending a task in `dispatch_task_to_worker()` at `controller.c:292`, the controller treats that worker as dead through `handle_worker_failure()` at `controller.c:262`, closes the pipe endpoints associated with it, and re-queues the unfinished task so that another live worker can process it later. If the worker reads EOF, a short read, or any other read error while expecting a `TaskMessage` in `worker_main_loop()` at `worker.c:45`, it exits with failure or normal termination depending on the case. If the worker receives a `TaskMessage` whose `type` field is not `MSG_TASK` or `MSG_SHUTDOWN`, it treats that as a protocol violation and exits.

Message Type: `MSG_SHUTDOWN`

Sender and receiver: Controller to worker.

Encoding: A fixed-size `TaskMessage` struct written in binary form in a single `write_full` call. The controller sets `type` to `MSG_SHUTDOWN` and uses `task_id = -1`. The `filename` field is left empty.

```
typedef struct {
    int32_t type;
    int32_t task_id;
    char filename[512];
} TaskMessage;
```

Semantics and invariants: This is a control message rather than a work request. It tells a worker that no more files remain to be processed and that it should terminate normally. The controller sends this message in `send_shutdown_to_workers()` at `controller.c:136`. The worker still reads exactly `sizeof(TaskMessage)` bytes before interpreting the message in `worker_main_loop()` at `worker.c:45`. After reading this message and checking that `type` is `MSG_SHUTDOWN`, the worker exits its main loop without sending a result.

Error handling: If the controller cannot write the shutdown message to a worker pipe in `send_shutdown_to_workers()` at `controller.c:136`, it prints a warning and marks that worker as no longer alive. Since shutdown is sent only after all tasks have already been completed or accounted for, a failed shutdown write does not lose any work. If a worker crashes before receiving the shutdown message, the controller eventually reaps it during cleanup through `reap_worker_if_possible()` at `controller.c:115`.

Message Type: MSG_RESULT

Sender and receiver: Worker to controller.

Encoding: A fixed-size `ResultMessage` struct written in binary form in a single `write_full` call. The struct contains the message type, task identifier, worker process ID, a success flag, line, word, and character counts, an error code, a fixed-size filename field, and a fixed-size error message field.

```
typedef struct {
    int32_t type;
    int32_t task_id;
    int32_t worker_pid;
    int32_t success;
    uint64_t line_count;
    uint64_t word_count;
    uint64_t char_count;
    int32_t error_code;
    char filename[512];
    char error_message[128];
} ResultMessage;
```

Semantics and invariants: This message reports the outcome of one previously assigned MSG_TASK. The worker constructs it in `process_task_message()` at `worker.c:14` by setting type to MSG_RESULT, copying back the original task_id, storing its process ID in worker_pid, and then filling either the line, word, and character counts for a successful analysis or the error_code and error_message fields for a failed analysis. The controller reads exactly one full `ResultMessage` from whichever result pipe poll reports as ready in `collect_ready_results()` at `controller.c:399`, verifies that the type field is MSG_RESULT, and checks that the task_id matches the task currently assigned to that worker in `mark_task_completed()` at `controller.c:350`. That invariant is important because the controller uses it to ensure that results are matched to the correct in-flight task before updating totals.

Error handling: If a worker cannot write a result message to its result pipe in `process_task_message()` at `worker.c:14`, it reports the failure with perror and exits with failure. On the controller side, if `read_full()` in `collect_ready_results()` at `controller.c:399` returns end-of-file, a short read, or a general read error, or if the received struct has the wrong type or an unexpected task_id, the controller treats the worker as failed through `handle_worker_failure()` at `controller.c:262`. In that case it closes the worker's pipes, re-queues any task that was still marked as running for that worker,

and continues as long as at least one live worker remains. If all workers die before all tasks are completed, the controller prints a fatal error and terminates from `run_controller()` at `controller.c:481`.

5.5 Concurrency Model

This project uses the Category 1 concurrency model of one parent controller process coordinating a fixed pool of child worker processes. The parent creates concurrency at startup by spawning all three workers before any file-analysis tasks are processed. This setup is implemented in `spawn_worker()` at `controller.c:182`, which first creates two pipes for each worker and then calls `fork` so that the child inherits the pipe file descriptors it needs. After the fork, the child keeps only the read end of its task pipe and the write end of its result pipe, while the parent keeps only the write end of the task pipe and the read end of the result pipe. Because `run_controller()` at `controller.c:481` creates all three children before entering the main dispatch loop, at least three workers exist concurrently during execution, satisfying the assignment requirement.

The parent does not use mutexes, signals, or shared memory to coordinate work. Instead, synchronization is provided entirely by pipe-based message passing and a small amount of controller state. A worker is considered ready when it is alive and its busy flag is clear. In practice, that means the parent has either just created the worker or has already received that worker's previous result and marked it idle again. The `dispatch_pending_tasks()` function at `controller.c:326` scans the worker table for idle workers and the task table for queued files, then sends one `TaskMessage` to each available worker through `dispatch_task_to_worker()` at `controller.c:292`. Once a task is assigned, the controller marks that worker as busy and records which task is currently in flight for that process.

Each worker runs a simple loop in `worker_main_loop()` at `worker.c:45`. It blocks in `read_full()` at `ipc.c:7` waiting for the next `TaskMessage` from the controller, processes the requested file through `process_task_message()` at `worker.c:14`, sends back exactly one `ResultMessage`, and then returns to the blocking read state. This design keeps the worker logic simple while still allowing multiple files to be analyzed at the same time in different processes. The blocking read on the task pipe is sufficient to synchronize each worker with the controller, because an idle worker naturally sleeps until new work arrives.

On the parent side, results are collected concurrently using `poll` in `collect_ready_results()` at `controller.c:399`. Before each wait, the controller builds an array of `pollfd` structures containing the read end of every live result pipe and asks `poll` to block until one or more workers either produce a result or disconnect. This avoids waiting on workers in a fixed order and ensures that the parent reacts to whichever worker finishes first. When `poll` reports input on a result pipe, the controller reads exactly one `ResultMessage`, verifies that it matches the expected in-flight task for that worker in `mark_task_completed()` at `controller.c:350`, records the result, marks the worker idle, and immediately makes that worker available for another queued task.

Child processes are collected with `waitpid` through `reap_worker_if_possible()` at `controller.c:115`. If a worker fails during execution, the controller closes its pipe

endpoints, re-queues any task that was still assigned to that worker, and calls `waitpid` with `WNOHANG` through `reap_worker_if_possible()` so that the dead child can be collected without blocking the rest of the program. During final cleanup, after the controller has sent `MSG_SHUTDOWN` to all remaining live workers in `send_shutdown_to_workers()` at `controller.c:136`, `cleanup_controller()` calls `waitpid` without `WNOHANG` through `reap_worker_if_possible()` to reap any children that are still running. This prevents zombie processes and ensures that all child processes are collected before the program exits.

5.6 Error Handling and Robustness

This project checks the return values of all important system calls and treats pipe failures, malformed messages, and file-processing errors as normal runtime conditions that must be handled explicitly. The design goal, implemented across `run_controller()` at `controller.c:481`, `collect_ready_results()` at `controller.c:399`, and `analyze_file()` at `analysis.c:22`, is that a single bad input file or a single worker failure should not immediately crash the entire program unless no workers remain able to make progress.

One bad behaviour is that a worker may fail or disconnect while it is processing a task. This can happen if the worker exits unexpectedly, crashes, or closes its result pipe before sending a complete reply. The `collect_ready_results()` function at `controller.c:399` detects this through poll events such as `POLLHUP` or `POLLERR`, or through `read_full()` at `ipc.c:7` returning end-of-file, a short read, or a general read error. The controller then calls `handle_worker_failure()` at `controller.c:262`, which marks the worker as dead, closes the worker's pipe file descriptors, re-queues any task that had been assigned to that worker, and attempts to reap the child with `waitpid` through `reap_worker_if_possible()` at `controller.c:115`. This allows the remaining live workers to continue processing the unfinished work.

A second bad behaviour is that the controller may receive malformed or inconsistent data from a worker. For example, a result message might have the wrong type field, an invalid task identifier, or a `task_id` that does not match the task currently recorded as running for that worker. The controller checks all of these conditions in `collect_ready_results()` at `controller.c:399` and `mark_task_completed()` at `controller.c:350` before accepting a result. If any of these invariants fail, the controller treats the result as a protocol error rather than silently accepting corrupted state. It marks the worker as failed through `handle_worker_failure()` at `controller.c:262`, closes the associated pipes, and re-queues the unfinished task if necessary. This prevents incorrect counts from being added to the final totals.

A third bad behaviour is failure while writing to a pipe. If the controller cannot send a `TaskMessage` to a worker because `write_full()` at `ipc.c:35` fails inside `dispatch_task_to_worker()` at `controller.c:292`, it assumes that the worker is no longer usable and handles it as a worker failure. If a worker cannot send a `ResultMessage` back to the controller because the result pipe has been closed or another write error occurs, `process_task_message()` at `worker.c:14` prints an error with `perror` and exits with failure. The program also ignores `SIGPIPE` in `main()` at `main.c:13` so that a broken pipe does not terminate the process abruptly; instead, the failed write is reported through the normal error path and handled explicitly.

A fourth bad behaviour is that an input file may be invalid or unreadable. For example, the file may not exist, the process may not have permission to open it, or a read error may occur while scanning the file. In `analyze_file()` at `analysis.c:22`, the code checks the return values of `open`, `read`, and `close`. If any of these operations fail, it stores the corresponding `errno` value and human-readable message in the `AnalysisResult` structure. The worker then places that information in a `ResultMessage` in `process_task_message()` at `worker.c:14` with `success` set to `false`, and the controller reports the file-specific error in `mark_task_completed()` at `controller.c:350` without terminating the rest of the program. This means one bad input file does not prevent other files from being analyzed successfully.

A fifth bad behaviour is an invalid filename that is too long to fit in the fixed-size protocol buffer. Before sending a task to a worker, `dispatch_task_to_worker()` at `controller.c:292` checks whether the filename fits inside the `TaskMessage` `filename` field. If it does not, the controller does not send a truncated or malformed message. Instead, `fail_task_locally()` in `controller.c` at line 278 marks the task as failed, records an `ENAMETOOLONG` error, increments the failed-task count, and continues processing the remaining files. This avoids corrupting the protocol while still giving the user a clear explanation of the failure.

The main limitation of the current implementation is that it does not spawn replacement workers after a crash. Instead, robustness comes from re-queuing unfinished work in `handle_worker_failure()` at `controller.c:262` and continuing with the surviving worker processes. If all workers die before all tasks have completed, `report_all_workers_dead_if_needed()` in `controller.c` at line 471 detects that no further progress is possible, and `run_controller()` at `controller.c:481` exits cleanly rather than hanging indefinitely.

5.7 Team Contributions

This project was completed entirely by me, Alim Javed Pathan.